
LAPORAN PRAKTIKUM INTERNET OF THINGS

Praktikum 4



Oleh:

NAMA : Haikal Ahnaf Yahya

NPM : 25782098

KELAS : TRI – 3D

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

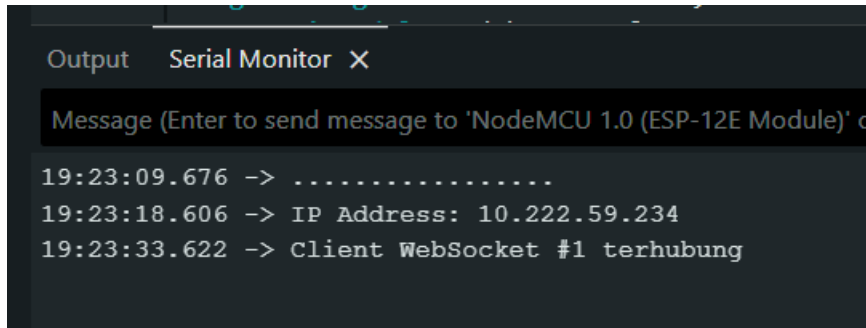
**TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG**

2026/2027

Modul 4: Komunikasi Asinkron dan WebSockets

4.5.1 Praktikum 1: *Real-Time Dashboard & Sinkronisasi Tombol Fisik*

- Output Serial Monitor



```
Output  Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' o
19:23:09.676 -> .....
19:23:18.606 -> IP Address: 10.222.59.234
19:23:33.622 -> Client WebSocket #1 terhubung
```

Serial Monitor menunjukkan bahwa ESP8266 berhasil terhubung ke jaringan WiFi dan mendapatkan alamat IP 10.22.59.234. Selanjutnya, terdapat pesan “Client WebSocket #1 terhubung” yang menandakan bahwa browser berhasil terhubung ke ESP8266 melalui WebSocket. Dengan demikian, koneksi WiFi dan komunikasi antara ESP8266 dengan halaman web telah berhasil dilakukan.

- Hasil Praktikum



Pengujian sistem Smart Room berhasil dilakukan. Sensor DHT dapat membaca suhu sebesar 31,0°C dan menampilkannya pada antarmuka web secara real-time. LED juga berhasil dikendalikan melalui halaman web, ditunjukkan dengan status LED: ON dan LED fisik yang menyala.

4.6 Latihan

1. Latihan 1 (Teori Jaringan): Jelaskan dengan kata-kata Anda sendiri, mengapa teknologi koneksi WebSockets jauh lebih hemat konsumsi data/bandwidth internet dan tidak membuat layar berkedip jika dibandingkan dengan metode Meta Refresh HTTP HTML yang Anda gunakan pada Tugas Praktikum Modul 3?

- Nggak Bikin Berkedip: Meta Refresh mengunduh ulang seluruh halaman web dari nol tiap beberapa detik, makanya layar kedip. WebSocket cuma ngirim data mentah yang berubah aja (misal: tulisan "ON"), jadi JavaScript tinggal ngubah bagian itu tanpa reload halaman.
- Lebih Hemat Bandwidth: Meta Refresh ngirim header HTTP berulang kali dan unduh file web terus-terusan. WebSocket cuma handshake sekali di awal, habis itu pengiriman data selanjutnya cuma butuh beberapa byte aja.

2. Latihan 2 (Analisis Waktu/Timer): Apa bahaya sistematis yang akan terjadi pada komunikasi WebSockets (reaksi antara browser dan NodeMCU) jika Anda bersikeras nekat memanggil sintaks `delay(3000)` di dalam siklus fungsi `void loop()` pada draf arsitektur kompleks ini?

Kalau nekat memakai `delay(3000)` di dalam `loop()`, ini dampak buruk yang bakal terjadi:

- a) Sistem Blocking: `delay(3000)` menghentikan proses selama 3 detik, sehingga tombol dan browser tidak bisa merespons.
- b) WebSocket Terputus: `delay()` membuat NodeMCU terlambat merespons sehingga koneksi WebSocket bisa mengalami timeout.
- c) Tidak Real-Time: Tombol fisik dan web menjadi lambat sehingga sinkronisasi tidak berjalan dengan baik.

3. Latihan 3 (Modifikasi Front-End): Analisislah fungsi JavaScript pada blok // Merombak UI Tombol & Teks sesuai status hardware. Nama ID elemen HTML (`<span id="...">`) apa saja yang menjadi target injeksi/pengubahan teks oleh script yang ditulis oleh Anda?

- `tempValue` target untuk mengubah nilai suhu. Dan `ledStatus` target untuk mengubah status LED (ON/OFF).

4. Latihan 4 (Pengembangan Telemetry Sensor - Kelembapan): Pada Praktikum 1, antarmuka web hanya menyajikan data suhu (temperature). Modifikasilah program agar sistem dapat membaca dan menampilkan data kelembapan (humidity) secara real-time via WebSockets:

- Sisi Front-End (HTML & JavaScript):

```
</div>

<div class="card">
  <h2>Kelembapan: <span id="humValue">--</span> %</h2>
</div>
```

```
}

if (dataObj.hum !== undefined) {
  document.getElementById('humValue').innerHTML = dataObj.hum;
}
```

- Sisi Back-End (C++):

```

// ...
void notifyClients() {
    String jsonString = "{\"led\": \"\" + String(ledState ? 1 : 0) + "\", ";
    jsonString += "\"suhu\": \"\" + currentTemp + "\", ";
    jsonString += "\"hum\": \"\" + currentHum + "\"}";
    ws.textAll(jsonString);
}

```

- Output Serial Monitor

```

16:25:55.598 -> Suhu: 31.8 C | Kelembapan: 52.6 %
16:26:00.056 -> Suhu: 31.7 C | Kelembapan: 53.0 %
16:26:05.046 -> .....
16:26:18.338 -> IP Address: 10.222.59.234
16:26:18.338 -> Suhu: 31.7 C | Kelembapan: 54.6 %
16:26:21.325 -> Suhu: 31.7 C | Kelembapan: 55.4 %
16:26:22.159 -> Client WebSocket #1 terhubung
16:26:24.351 -> Suhu: 31.8 C | Kelembapan: 55.2 %
16:26:27.370 -> Suhu: 31.8 C | Kelembapan: 55.9 %
16:26:29.244 -> Client WebSocket #2 terhubung

```

Sistem berhasil membaca suhu sekitar 31,7–31,8°C dan kelembapan sekitar 52,6–55,9% secara berkala. IP Address ESP8266 yang digunakan adalah 10.222.59.234.

- Hasil Praktikum



Hasilnya pada web server terdapat penambahan Kelembapan: 53,5%